Reviewer Pack — Minimal Local Ring Unit Group Size and Circuit Monotone Widt…

Entry 08a0b0a0165c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 21:32:19 UTC

Minimal Local Ring Unit Group Size and Circuit Monotone Width Correlation via Frobenius Endomorphism

Entry ID: `08a0b0a0165c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 21:32:19 UTC

1. Conjecture

Field A (mathematical branch): Local Ring Theory **Field B** (complexity object): Boolean Circuits

Statement:

> For every Boolean circuit C with n inputs, the minimal size of the unit group in the local ring associated with its negacyclic representation is linearly correlated with its monotone width, such that log|U(C)| = Θ(w_mon(C)), where |U(C)| is the order of the unit group and w_mon(C) is the monotone width of the circuit.

Rationale (proposer's reasoning):

> The Frobenius endomorphism provides a way to relate polynomial functions over finite fields to the structure of circuits, which may expose hidden complexity-theoretic properties through local ring analysis. The correlation between unit group size and monotone width could reveal new insights into the inherent difficulty of circuit satisfiability problems.

Taxonomy category: `LOCAL_RING_THEORY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9fd93ae98f13dafa`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if Pearson correlation coefficient ≥ 0.8 AND metric mean ≤ 3 standard deviations from the expected value, across at least 1000 randomly generated circuits.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"local ring theory" AND "Boolean circuits" AND "Frobenius endomorphism"` - `"unit group size" IN local ring theory AND "circuit monotone width"` - `"negacyclic representation" AND "order of unit group" AND "monotone width correlation"`

Top relevant hits considered: - [http://arxiv.org/abs/1607.05594v2] Poincaré series of compressed local Artinian rings with odd top socle degree - [http://arxiv.org/abs/astro-ph/0405128v1] Probing the Evolution of the Galaxy Interaction/Merger Rate Using Collisional Ring Galaxies - [http://arxiv.org/abs/2506.06785v1] Extending dependencies to the taggedPBC: Word order in transitive clauses

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=2.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        if n == 1:
            return [random.choice([0, 1])]
        else:
            left = generate_circuit(n // 2)
            right = generate_circuit(n - n // 2)
            return [random.choice([0, 1]) for _ in range(n)] + left + right

    def compute_local_ring(circuit):
        n = len(circuit)
        ring = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(n + 1):
            ring[i][i] = 1
        for i in range(n):
            if circuit[i] == 1:
                for j in range(i, n + 1):
                    ring[j][j - 1] += ring[j][i]
                    ring[j][i] -= ring[j][i - 1]
        return ring

    def measure_unit_group_size(ring):
```

```
        n = len(ring) - 1
        unit_group_size = 0
        for i in range(n + 1):
            if all(ring[i][j] == 0 for j in range(i, n + 1)):
                unit_group_size += 1
        return unit_group_size

    def measure_monotone_width(circuit):
        n = len(circuit)
        width = 0
        for i in range(n):
            if circuit[i] == 1:
                width += 1
        return width

    def pearson_correlation(x, y):
        n = len(x)
        mean_x = sum(x) / n
        mean_y = sum(y) / n
        cov_xy = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n))
        std_x = math.sqrt(sum((x[i] - mean_x) ** 2 for i in range(n)) / n)
        std_y = math.sqrt(sum((y[i] - mean_y) ** 2 for i in range(n)) / n)
        if std_x == 0 or std_y == 0:
            return 0
        return cov_xy / (std_x * std_y)

    unit_group_sizes = []
    monotone_widths = []

    for _ in range(100):
        circuit = generate_circuit(random.randint(5, 40))
        ring = compute_local_ring(circuit)
        unit_group_size = measure_unit_group_size(ring)
        monotone_width = measure_monotone_width(circuit)
        unit_group_sizes.append(unit_group_size)
        monotone_widths.append(monotone_width)

    correlation = pearson_correlation(unit_group_sizes, monotone_widths)
    mean_value = sum(unit_group_sizes) / len(unit_group_sizes)
    std_value = math.sqrt(sum((x - mean_value) ** 2 for x in unit_group_sizes) / len(unit_group_si

    return {
        "metric_name": "Pearson Correlation",
        "metric_value": correlation,
        "instances_tested": len(unit_group_sizes),
        "n_max": max(len(circuit) for _ in range(100)),
        "conjecture_holds": abs(correlation) >= 0.8 and abs(correlation - mean_value) <= 3 * std_v
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
```

```
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conje
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
ed': 100, 'n_max': 129, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 99.9898605138246, 'instances_tested':
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 99.98986871806719, 'instances_tested':
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 99.990996473974, 'instances_tested': 1
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 99.99080970696764, 'instances_tested':
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 99.98934840305579, 'instances_tested':
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 99.98981006873403, 'instances_tested':
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 99.98927297927962, 'instances_tested':
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 99.99018581552596, 'instances_tested':
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 99.9904798745062, 'instances_tested':
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 99.99085068552887, 'instances_tested':
TRIAL: {'metric_name': 'Pearson Correlation', 'metric_value': 99.99045278658272, 'instances_tested':
RESULT: SUPPORTED mean=99.99020311043161 std=0.0007246726533607731 support_fraction=1.0
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> The test code uses a random circuit generator that may not produce circuits with the properties necessary to validate the conjecture. The minimal size of the unit group in the local ring and monotone width are computed for these random circuits, which might not be representative of all possible circuits.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Safety rail: critic_challenge | original judge: The test results show a Pearson correlation coefficient of 99.99020311043161, which meets the pre-registered support condition of ≥ 0.8, and the metri | next: Further investigation into the properties of the random circuit generator used in the test to ensure it produces circuits that are representative of all possible circuits.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18230
2	preregistration	ollama_remote	glm4:latest	0	0	9000
3	novelty	ollama_remote	glm4:latest	0	0	8469
4	novelty	ollama_remote	glm4:latest	0	0	9442
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	91502
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30709
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12424
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24717
9	critic	ollama_remote	glm4:latest	0	0	11591
10	judge	ollama_remote	glm4:latest	0	0	9327

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 225410 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/08a0b0a0165c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/08a0b0a0165c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/08a0b0a0165c.tar.gz` (if generated)